

CO4 AT1

NAME:-S.THARUN KAMALESH

REG NO:-192473004

Q1. Transportation and Traffic Analysis

A city transportation department maintains data containing vehicle type, route, date, travel time, distance, traffic volume, and location. Explore the dataset, create groups based on vehicle type and route, identify outliers in travel time and traffic volume, and analyze relationships between distance, traffic volume, and travel time. Use time-related visualizations to identify traffic patterns and, where geographical information is available, represent important findings using maps. Prepare a report explaining how the results can support traffic management.

Python / Pandas Code

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("transportation.csv")
print(df.head())
print(df.shape)
print(df.info())
print(df.describe())
print(df.isnull().sum())

df['date'] = pd.to_datetime(df['date'])

vehicle_analysis = df.groupby('vehicle_type').agg(
    total_travel_time=('travel_time', 'sum'),
    average_travel_time=('travel_time', 'mean'),
    total_traffic=('traffic_volume', 'sum'))
print(vehicle_analysis)

route_analysis = df.groupby('route').agg(
    average_travel_time=('travel_time', 'mean'),
    average_distance=('distance', 'mean'),
    average_traffic=('traffic_volume', 'mean'))
print(route_analysis)

Q1 = df['travel_time'].quantile(0.25)
Q3 = df['travel_time'].quantile(0.75)
IQR = Q3 - Q1
outliers = df[(df['travel_time'] < Q1 - 1.5*IQR) |
               (df['travel_time'] > Q3 + 1.5*IQR)]
print("Travel time outliers:")
print(outliers)

print(df[['distance', 'traffic_volume', 'travel_time']].corr())

daily_traffic = df.groupby('date')['traffic_volume'].sum()
daily_traffic.plot()
plt.title("Traffic Volume Over Time")
plt.xlabel("Date")
plt.ylabel("Traffic Volume")
plt.show()

route_analysis['average_travel_time'].plot(kind='bar')
plt.title("Average Travel Time by Route")
plt.xlabel("Route")
plt.ylabel("Average Travel Time")
plt.show()
```

Google Colab Output Screenshot

Output Screenshot

Out[1]:

vehicle_type	total_travel_time	average_travel_time	total_traffic
Bike	60	20.00	300
Bus	142	47.33	570
Car	178	44.50	685
Truck	118	59.00	380

Route summary			
route	average_travel_time	average_distance	average_traffic
A	33.00	11.00	150.00
B	34.75	13.25	143.75
C	56.75	20.50	190.00

Out[2]:

Correlation			
	distance	traffic_volume	travel_time
distance	1.000	0.871	0.960
traffic_volume	0.871	1.000	0.956
travel_time	0.960	0.956	1.000

Q2. Food Delivery Analysis

A food delivery platform has collected order information including customer location, restaurant, food category, order value, delivery time, order date, and customer rating. Analyze the dataset by exploring its structure, grouping orders by restaurant and food category, identifying unusual delivery times, and determining whether order value is related to customer ratings or delivery time. Analyze changes in orders over time and present the important findings using charts and other suitable visualizations.

Python / Pandas Code

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("food_delivery.csv")
print(df.head())
print(df.shape)
print(df.info())
print(df.describe())
print(df.isnull().sum())

df['order_date'] = pd.to_datetime(df['order_date'])

restaurant_analysis = df.groupby('restaurant').agg(
    total_orders=('restaurant', 'count'),
    total_order_value=('order_value', 'sum'),
    average_delivery_time=('delivery_time', 'mean'),
    average_rating=('customer_rating', 'mean'))
print(restaurant_analysis)

category_analysis = df.groupby('food_category').agg(
    total_orders=('food_category', 'count'),
    average_order_value=('order_value', 'mean'),
    average_delivery_time=('delivery_time', 'mean'),
    average_rating=('customer_rating', 'mean'))
print(category_analysis)

Q1 = df['delivery_time'].quantile(0.25)
Q3 = df['delivery_time'].quantile(0.75)
IQR = Q3 - Q1
delivery_outliers = df[(df['delivery_time'] < Q1 - 1.5*IQR) |
    (df['delivery_time'] > Q3 + 1.5*IQR)]
print("Unusual delivery times:")
print(delivery_outliers)

print(df[['order_value', 'delivery_time', 'customer_rating']].corr())

orders_over_time = df.groupby('order_date').size()
orders_over_time.plot()
plt.title("Orders Over Time")
plt.xlabel("Order Date")
plt.ylabel("Number of Orders")
plt.show()

restaurant_analysis['average_delivery_time'].plot(kind='bar')
plt.title("Average Delivery Time by Restaurant")
plt.xlabel("Restaurant")
plt.ylabel("Delivery Time")
plt.show()
```

Output Screenshot

Out[1]:

restaurant				
	total_orders	total_order_value	average_delivery_time	average_rating
R1	4	1470	33.75	4.38
R2	4	1700	48.75	4.00
R3	4	1710	46.25	4.18

Food category summary				
	total_orders	average_order_value	average_delivery_time	average_rating
food_category				
Burger	3	333.33	38.33	4.20
Chinese	3	470.00	55.00	3.83
Pizza	3	533.33	51.67	4.00
South Indian	3	290.00	26.67	4.70

Out[2]:

Correlation			
	order_value	delivery_time	customer_rating
order_value	1.000	0.843	-0.810
delivery_time	0.843	1.000	-0.963
customer_rating	-0.810	-0.963	1.000

Q3. Movie and Entertainment Analysis

A streaming platform has collected information about movies and shows, including title, genre, release year, duration, language, rating, number of viewers, and country. Explore and analyze the dataset, create groups based on genre and language, identify relationships between ratings, duration, and viewership, and detect unusual values. Present trends across release years using time-related charts and use appropriate visualizations to identify the most popular genres and content categories.

Python / Pandas Code

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("movies.csv")
print(df.head())
print(df.shape)
print(df.info())
print(df.describe())
print(df.isnull().sum())

genre_analysis = df.groupby('genre').agg(
    total_movies=('title', 'count'),
    average_rating=('rating', 'mean'),
    average_duration=('duration', 'mean'),
    total_viewers=('viewers', 'sum'))
print(genre_analysis)

language_analysis = df.groupby('language').agg(
    total_movies=('title', 'count'),
    average_rating=('rating', 'mean'),
    total_viewers=('viewers', 'sum'))
print(language_analysis)

print(df[['rating', 'duration', 'viewers']].corr())

Q1 = df['rating'].quantile(0.25)
Q3 = df['rating'].quantile(0.75)
IQR = Q3 - Q1
rating_outliers = df[(df['rating'] < Q1 - 1.5*IQR) |
    (df['rating'] > Q3 + 1.5*IQR)]
print("Rating outliers:")
print(rating_outliers)

year_analysis = df.groupby('release_year').size()
year_analysis.plot()
plt.title("Movies Released by Year")
plt.xlabel("Release Year")
plt.ylabel("Number of Movies")
plt.show()

genre_analysis['total_viewers'].sort_values().plot(kind='barh')
plt.title("Viewership by Genre")
plt.xlabel("Total Viewers")
plt.ylabel("Genre")
plt.show()

plt.scatter(df['rating'], df['viewers'])
plt.title("Rating vs Viewership")
```

Output Screenshot

Out[1]:

	total_movies	average_rating	average_duration	total_viewers
genre				
Action	4	4.70	137.5	740000
Comedy	3	4.27	105.0	340000
Drama	3	4.47	140.0	485000
Thriller	2	3.95	131.0	155000

Language summary			
	total_movies	average_rating	total_viewers
language			
English	5	4.56	865000
Hindi	3	4.10	285000
Tamil	4	4.45	570000

Out[2]:

Correlation			
	rating	duration	viewers
rating	1.000	0.452	0.927
duration	0.452	1.000	0.590
viewers	0.927	0.590	1.000

Q4. Agriculture and Crop Analysis

An agricultural department has collected data containing crop type, district, cultivated area, rainfall, fertilizer usage, and crop yield. Import and explore the data, group the records by crop and district, identify outliers in yield and fertilizer usage, and analyze correlations between rainfall, cultivated area, fertilizer usage, and crop yield. Use charts and maps, where geographical information is available, to communicate the major findings and suggest suitable strategies for improving agricultural productivity.

Python / Pandas Code

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("agriculture.csv")
print(df.head())
print(df.shape)
print(df.info())
print(df.describe())
print(df.isnull().sum())

crop_analysis = df.groupby('crop_type').agg(
    total_area=('cultivated_area', 'sum'),
    average_rainfall=('rainfall', 'mean'),
    average_fertilizer=('fertilizer_usage', 'mean'),
    average_yield=('crop_yield', 'mean'))
print(crop_analysis)

district_analysis = df.groupby('district').agg(
    total_area=('cultivated_area', 'sum'),
    average_rainfall=('rainfall', 'mean'),
    average_fertilizer=('fertilizer_usage', 'mean'),
    average_yield=('crop_yield', 'mean'))
print(district_analysis)

Q1 = df['crop_yield'].quantile(0.25)
Q3 = df['crop_yield'].quantile(0.75)
IQR = Q3 - Q1
yield_outliers = df[(df['crop_yield'] < Q1 - 1.5*IQR) |
    (df['crop_yield'] > Q3 + 1.5*IQR)]
print("Yield outliers:")
print(yield_outliers)

Q1 = df['fertilizer_usage'].quantile(0.25)
Q3 = df['fertilizer_usage'].quantile(0.75)
IQR = Q3 - Q1
fertilizer_outliers = df[(df['fertilizer_usage'] < Q1 - 1.5*IQR) |
    (df['fertilizer_usage'] > Q3 + 1.5*IQR)]
print("Fertilizer outliers:")
print(fertilizer_outliers)

print(df[['rainfall', 'cultivated_area', 'fertilizer_usage', 'crop_yield']].corr())

crop_analysis['average_yield'].plot(kind='bar')
plt.title("Average Crop Yield by Crop Type")
plt.xlabel("Crop Type")
plt.ylabel("Average Yield")
plt.show()
```

Output Screenshot

Out[1]:

	total_area	average_rainfall	average_fertilizer	average_yield
crop_type				
Cotton	155	610.00	44.50	215.00
Maize	290	696.67	49.33	250.00
Rice	570	862.50	60.25	357.50
Wheat	335	686.67	51.00	296.67

District summary				
	total_area	average_rainfall	average_fertilizer	average_yield
district				
A	565	732.0	52.60	290.0
B	440	727.5	52.25	280.0
C	345	750.0	53.00	310.0

Out[2]:

Correlation				
	rainfall	cultivated_area	fertilizer_usage	crop_yield
rainfall	1.000	0.920	0.964	0.900
cultivated_area	0.920	1.000	0.978	0.968
fertilizer_usage	0.964	0.978	1.000	0.944
crop_yield	0.900	0.968	0.944	1.000